

JOURNAL DE BORD

Cryptographie : collisions de md5

Projet intégratif Sujet 1

SAE 204

Séance 1	1
Séance 2	2
Séance 3	3
Séance 4	4
Séance 5	5
Séance 6	9
Séance 7	

Séance 1

DATE : 12/05/2026 --- **Heure :** 12h18 --- **Séance :** 1

Étudiant présent : LACOSTE Nathan

Travail réalisé : Recherche et documentation

// J'espère que vous savez bien parler anglais car beaucoup des documents que j'ai consulté sont en anglais.

Découverte Hachage / md5 :

- https://www.youtube.com/watch?v=R_mOWu3s6y4 (Anglais)
- <https://www.youtube.com/watch?v=5MiMK45gkTY&t=53s> (Anglais)
- <https://www.youtube.com/watch?v=jSpEPpQlD10> (Français)
- https://fr.wikipedia.org/wiki/Fonction_de_hachage_cryptographique (Français)
- <https://www.okta.com/fr-fr/identity-101/md5/> (Français)

Papier de recherche (thèse) :

- <https://marc-stevens.nl/research/hashclash/On%20Collisions%20for%20MD5%20-%20M.M.J.%20Stevens.pdf> (Anglais)

Découverte CUDA :

- <https://www.youtube.com/watch?v=pPStdjuYzSI> (Anglais)
- https://fr.wikipedia.org/wiki/Compute_Unified_Device_Architecture (Français)
- <https://www.youtube.com/watch?v=r9lqwpMR9TE> (Anglais)

Documentation C++ :

- <https://www.w3schools.com/CPP/default.asp> (Anglais)
- Code Giraphe *// Code du projet des Olympiade de l'ingé que je bidouille pour pas trop perdre la main en C++*

Séance 2

DATE : 12/05/2026 --- Heure : 17h38 --- Séance : 2

Étudiant présent : LACOSTE Nathan

Travail réalisé : Entraînement code C++

// lien vers site utilisé pour les exercices :

// https://fr.wikibooks.org/wiki/Exercices_en_langage_C%2B%2B/Notions_de_base

Exercice réaliser :

- Exercice 1 `Hello world !`
ce code utilise cout pour afficher des trucs

```
=== Code Execution Successful ===
```

- Exercice 2


```
#include <iostream>
using namespace std;

int main() {
    // Write C++ code here
    int longueur;
    int largeur;
    string choix;

    cout << "longueur du champs (en mètres) : ";
    cin >> longueur;
    cout << "largeur du champs (en mètres) : ";
    cin >> largeur;
    cout << "a : périmètre b : surface\n" << "votre choix : ";
    cin >> choix;

    if (choix == "a") {
        int perimetre = (largeur + longueur)*2;
        cout << "périmètre du champs : " << perimetre << " mètres";
    }
    else {
        int surface = largeur * longueur;
        cout << "surface du champs : " << surface << " mètres carrés";
    }
    return 0;
}
```

longueur du champs (en mètres) : 10
largeur du champs (en mètres) : 15
a : périmètre b : surface
votre choix : b
surface du champs : 150 mètres carrés

=== Code Execution Successful ===

- Exercice 3 :

```
1 // Online C++ compiler to run C++ program online
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // Write C++ code here
7     int num;
8     double somme = 0 ;
9     int oly;
10
11     cout << "Nombre de notes : ";
12     cin >> oly;
13     cout << "\nInserer les numéros un à un :\n";
14     for (int ayo = 1; ayo <= oly; ayo++) {
15         cout << "note " << ayo << " : ";
16         cin >> num; somme = somme + num;
17     }
18
19     double moy = somme / oly;
20     cout << "la moyenne est : " << moy;
21     return 0;
22 }
```

Nombre de notes : 7

Inserer les numéros un à un :

note 1 : 15
note 2 : 20
note 3 : 16
note 4 : 18
note 5 : 14
note 6 : 17
note 7 : 13

la moyenne est : 16.1429

=== Code Execution Successful ===

Séance 3

DATE : 20/05/2026 --- **Heure :** 17h41 --- **Séance :** 3

Étudiant présent : LACOSTE Nathan BELHARET ADAM

Travail réalisé : Galère

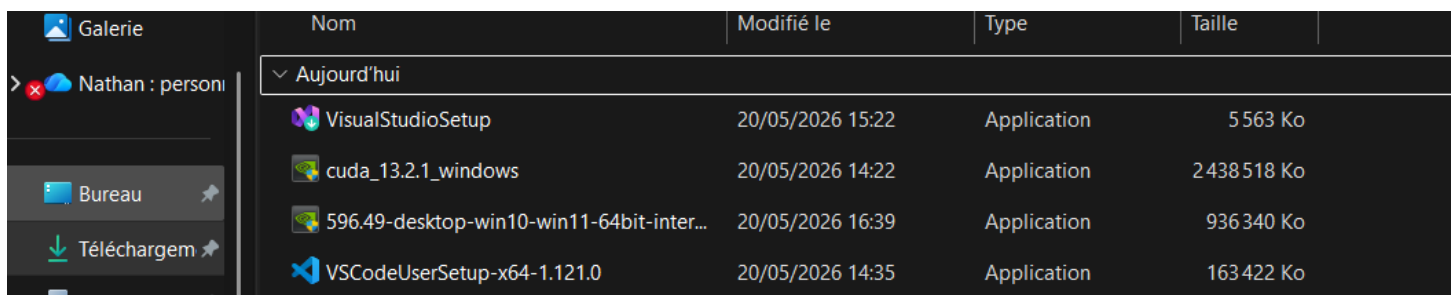
Nous avons tout essayé pour faire marcher cette carte graphique.

Nous avons d'abord tenté d'installer VSCode sur l'ordinateur de Nathan (aka mon ordi), mais l'éditeur était beaucoup trop complexe à prendre en main, avec des documentations compliquer.

Finalement après pas mal de recherche la doc nous a renvoyer vers Visual Studio. Que nous avons donc installé, cet IDE était plus facile à comprendre cependant un autre problème as vite émerger.

Mon ordinateur ainsi que le poste présent dans la salle ne détectaient pas la carte graphique, nous avons donc fait encore plus de recherche pour trouver une solution. Solution que nous pensions avoir trouver quand un site nous disait qu'il s'agissait d'un problème de drivers. Nous avons donc téléchargé les drivers nécessaire, mais la carte n'étant pas détecter par les ordis, impossible pour les drivers de s'installer.

Après cet n-ième échec, nous avons donc décider de demander à Chatgpt quel était le problème et il s'avère que mon ordi n'est pas compatible Thunderbolt, et que pour le poste présent dans la salle, c'est un problème lié au porte PCIE.



Nom	Modifié le	Type	Taille
Aujourd'hui			
VisualStudioSetup	20/05/2026 15:22	Application	5 563 Ko
cuda_13.2.1_windows	20/05/2026 14:22	Application	2 438 518 Ko
596.49-desktop-win10-win11-64bit-inter...	20/05/2026 16:39	Application	936 340 Ko
VSCodeUserSetup-x64-1.121.0	20/05/2026 14:35	Application	163 422 Ko

//Copie d'écran de mes téléchargements afin que vous ayez une idée du temps que nous avons passé à galérer avant de céder de demander à chatgpt.

Séance 4

DATE : 20/05/2026 --- **Heure :** 11h58 --- **Séance :** 4

Étudiant présent : LACOSTE Nathan BELHARET ADAM

Travail réalisé : Codage

VICTOIRE !! Adam a réussi à faire marcher la carte graphique. Le problème était simple : le port de mon ordi n'est pas du thunderbolt donc la carte graphique n'était pas reconnue. *//un problème si simple nous a coûté une après-midi entière T-T*

Après avoir branché sur un ordinateur de l'itut (qui a donc un port thunderbolt), la carte graphique était bien reconnue.

Nous avons dû procéder à l'installation des différents logiciels/drivers pour le bon déroulé du projet.

Voici la liste des logiciels/drivers téléchargés :

- Studio Code Community 2022
- Cuda Toolkit 12.9
- Drivers de la carte graphique

Suite à cela, nous avons créé un nouveau projet sur Studio Code que l'on a nommé MD5-PROJET.

Séance 5

DATE : 29/05/2026 --- **Heure :** 16h59 --- **Séance :** 5

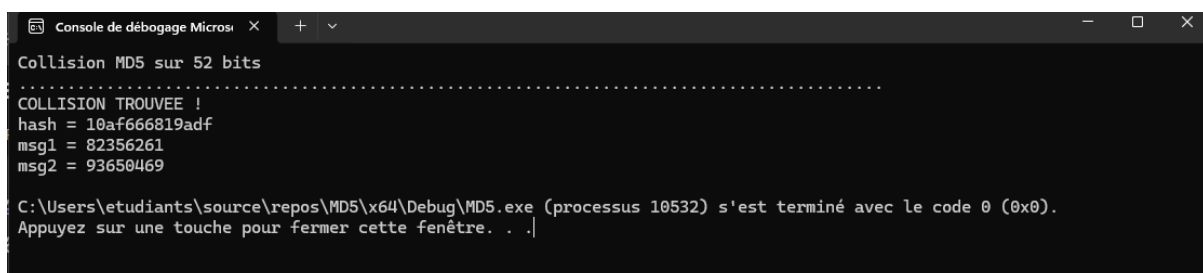
Étudiant présent : LACOSTE Nathan BELHARET ADAM

Travail réalisé : codage collision (64 bits) + recherches infos

Il faut savoir qu'il n'existe pas de bibliothèque md5 sur cuda, c'est à dire qu'on peut pas faire md5("message") et recevoir comme par magie le haché de ce "message". Il faut donc implémenter MD5 en GPU. Après des recherches Nathan a trouvé sur Wikipedia un code partiel de md5 en C++, chose qui nous a bien aidé car c'est la chose qui aurait prit le plus de temps.

Le reste du code est dédié aux attributions des thread, chaque thread aura un hash à calculer, à la gestion de la ram, car après que le hash soit calculé, ce dernier est envoyé au CPU et stocké dans la RAM pour ensuite comparé les différents hash qu'il a stocké et chercher si une collision a été trouver.

Par brute force nous avons réussi à trouver une collision pour maximum 52 bits, à voir si une collision entre 2 messages est possible sur 64 bits en brute force si nous le laissons tourner plusieurs heures d'affiler.



```
Console de débogage Micros...
Collision MD5 sur 52 bits
.....
COLLISION TROUVEE !
hash = 10af666819adf
msg1 = 82356261
msg2 = 93650469

C:\Users\etudiants\source\repos\MD5\x64\Debug\MD5.exe (processus 10532) s'est terminé avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .|
```

Pour continuer, nous avons fait des recherches pour trouver une méthode de collision sur les 128 bits, que nous détaillerons à la prochaine séance.

Pour finir nous avons commencé à travailler sur un code de reverse md5 par dictionnaire, c'est à dire que nous entrons le hash d'un mot de passe. Et notre code regarde dans un dictionnaire (dans notre cas c'est un dictionnaire trouvé sur GitHub issu d'une fuite de donnée de 2009 il me semble).

Les recherches de cette séance était porté sur qu'est-ce qu'un thread, pourquoi utiliser une carte graphique ainsi que qu'est-ce qu'est md5 et quels sont ses problèmes.

Md5 et ses problèmes.

Avant toute chose il est nécessaire de présenter md5. Le MD5, pour Message digest 5, est une fonction de hachage cryptographique qui permet d'obtenir l'empreinte numérique d'un fichier. Bien que cet algorithme présente un intérêt historique important, il est aujourd'hui considéré comme dépassé et absolument impropre à toute utilisation en cryptographie ou en sécurité. Il reste cependant encore utilisé pour vérifier l'intégrité d'un fichier après un téléchargement.

Le problème majeur de cet algorithme, qui le rend impropre à toute utilisation à des fins de sécurité sont ce que l'on appelle les "collision de hachage". Une collision de hachage survient lorsque deux entrées différentes créent la même valeur de hachage. La sécurité et le chiffrement d'un tel algorithme reposant sur la génération de valeurs de hachage uniques, les collisions représentent donc des failles de sécurité susceptibles d'être exploitées.

En effet les cybercriminels peuvent créer des collisions qui enverront alors une signature numérique qui sera acceptée par le destinataire. Même s'il ne s'agit pas de l'expéditeur réel, la collision fournit la même valeur de hachage, de sorte que le message du cybercriminel sera vérifié et accepté comme légitime.

Pourquoi utiliser une carte graphique ?

Bien que le hachage md5 soit extrêmement rapide à réaliser la méthode que nous utilisons (brute force) demande de réaliser des dizaines ou centaines de milliers de combinaisons différentes afin de réaliser une collision. C'est pourquoi la capacité de traitement des cartes graphiques (GPU) sont très intéressantes.

En effet, les GPU excellent dans le traitement parallèle grâce à des centaines, voire des milliers de cœurs, qui sont à l'origine de leurs capacités de traitement ultra-rapides.

//Par exemple la Geforce rtx 3060 Ti, que nous utilisons à 4864 cœurs CUDA.

Qu'est-ce qu'un thread ?

Un thread est la plus petite unité de carte graphique. En fait il s'agit d'une instruction que la carte graphique va multiplier des dizaines de milliers de fois. Cependant le gpu ne lance pas chaque thread un à un, il lance des warp qui sont un regroupement de 32 thread réalisant tous la même instruction au même moment. Mais

puisque il y a un ou plusieurs milliers de warp, il sont eux même regrouper en bloc de 32 warp.

//Donc c'est thread → warp → bloc

Bien sur toutes cette information ne sorte pas de nulle part, voici donc mes sources :

https://www.youtube.com/watch?v=OejS_7hlptw

<https://www.okta.com/fr-fr/identity-101/md5/>

<https://www.cybertek.fr/blog/composants/processeur/quel-processeur-pour-rtx-3060ti>

<https://aws.amazon.com/fr/what-is/gpu/>

<https://www.ibm.com/fr-fr/think/topics/gpu>

[Vulnerability Note VU#836068 MD5 vulnerable to collision attacks](#)

[Creating a rogue CA certificate](#)

Séance 6

DATE : 04/06/2026 --- **Heure :** 13h30 --- **Séance :** 4

Étudiant présent : LACOSTE Nathan BELHARET ADAM

Travail réalisé : Amélioration codage + Reverse MD5 brute force.

Cette journée de projet a porté sur l'amélioration du premier codage de recherche de collision md5. Notre première version utilisait le CPU ce qui créait un goulot d'étranglement, c'est-à-dire que le CPU n'était pas assez performant pour pouvoir utiliser la pleine puissance de la carte graphique dans la recherche de collision. Nous avons donc décidé de stocker la table de hachage non plus dans le CPU mais dans la CG directement.

Ce changement nous a permis de trouver une collision sur 64 bits en 220 secondes approximativement, contrairement à la première version du code où on n'arrivait même pas à trouver une collision md5 sur 64 bits pendant plusieurs minutes.

```
Console de débogage Micros... X + -
Collision MD5 sur 64 bits (table GPU de 2^28 slots)

Hashs testés : 49392123904 | Temps : 220.3 s

COLLISION TROUVEE !
hash = 1240c442abc725ea
msg1 = 110558666
msg2 = 49379194452
Temps total : 220.34 secondes

C:\Users\etudiants\source\repos\md5_bruite_force_collision\x64\Debug\md5_bruite_force_collision.exe (processus 20756) s'
est terminé avec le code 0 (0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .
```

Nous sommes donc à la moitié de notre but qui est de recréer une collision sur 128 bits. Ce sera le sujet de la prochaine séance.

En outre, nous avons commencé à étudier sur le reverse MD5. Pour cela, nous avons commencé avec quelque chose de basique. Nous avons fait un exercice en utilisant une base de données trouver (mots de passe) sur github suite à une fuite de données datant de 2009.

```
Console de débogage Micros... X + -
Hash cible : ae43792485331fa311a50a7972118adc
Dictionnaire : C:/Users/etudiants/Desktop/rockyou.txt

TROUVE : hello95
Tentatives : 262144

C:\Users\etudiants\source\repos\Reverse MD5\x64\Release\Reverse MD5.exe (processus 11672) s'est terminé avec le code 0 (
0x0).
Appuyez sur une touche pour fermer cette fenêtre. . .|
```

Ici, nous avons entré le hash d'un mot de passe (hello95), et il a bien retrouvé le mot de passe correspondant à ce hash. Cependant, cette méthode est quand même assez limitée car il y a une limite de mot de passe, à voir si il est possible d'utiliser plusieurs fichiers texte.

Séance 7, 8, 9

DATE : 05/06/2026 et 10/06/206 --- **Heure :** X --- **Séance :** 7,8,9

Étudiant présent : LACOSTE Nathan BELHARET ADAM

Travail réalisé : Reverse Brute force + étude temps collision sur 128 bits + reverse md5 hash artificiel

Après le petit exercice de reverse md5 à l'aide d'un dictionnaire notre but ici était de reproduire le brute force mais pour cette fois ci le reverse md5. Concernant le reverse md5 nous allons chercher un code sur internet mais c'est assez compliqué d'en trouver un "simple" contrairement à celui du dictionnaire qui était assez simple à trouver et un minimum compréhensif. Nous avons pu en tester quelques-uns qui étaient assez efficaces mais pour être honnête on ne comprenait pas grand-chose au code. Nous allons donc faire des recherches chez nous afin d'avoir un code assez "basique" sur lequel on pourra débiter le reverse md5 brute force pour ensuite faire du reverse md5 sur un hash artificiel.

Puisque à la séance 6 nous avons réussi à trouver un hash sur 64 bits en approximativement 220 secondes. Il serait intéressant de déterminer le temps qu'il nous faudrait afin de trouver 2 hash identiques par brute force

Pour trouver une collision sur 128 bits, soit $2^{n/2}$ (divisé par 2 car on ne recherche pas qu'un seul hash mais deux hash identiques) d'essais.

On a réussi à avoir une collision en 64 bits sur 220 secondes, ainsi nous faisons le calcul suivant : $2^{32} / 220 =$ environ 19.522.000 hash/secondes.

Notre cg calcul ce nombre de hash par secondes.

Pour avoir le temps de trouver une collision sur 128 bits nous faisons $2^{(n/2)} / 19522000$ soit 944 920 811 070 secondes. Ce qui nous donne environ 29000 années.